Data Structures - Recitation 4

Friday, September 30, 2022

The intuition behind recursion

- Given the solution to a smaller sub-problem, can we solve the original problem?
- Smaller sub-problem: The same problem but with a smaller input which is often the subset of the original input.
- Example:
  - Problem: find the minimum of the array: 101, 53, 23, 65, 28, 97, 14, 83
  - Sub-problems:
    - Find the minimum of the array: 101, 53, 23, 65, 28, 97, 14
    - Find the minimum of the array: 23, 65, 28, 97, 14, 83
    - Find the minimum of the array: 101, 53, 23, 65

Intuition behind recursion

- Given the minimum of the array a_sub = 101, 53, 23, 65, 28, 97, 14 is **x**; Can we find the minimum of a = 101, 53, 23, 65, 28, 97, 14, 83 ?
  - Yes. Integer.min(x, 83)
  - => minArray(a) = min(minArray(a_sub), 83)
  - => minArray(a) = min(minArray(a[0,....,n-2]), a[n-1])
    - Is this correct?

Intuition behind recursion

- Given the minimum of the array a_sub = 101, 53, 23, 65, 28, 97, 14 is **x**; Can we find the minimum of a = 101, 53, 23, 65, 28, 97, 14, 83 ?
  - Yes. Integer.min(x, 83)
  - => minArray(a) = min(minArray(a_sub), 83)
  - => minArray(a) = min(minArray(a[0,....,n-2]), a[n-1])
    - Is this correct? No
    - This is only correct when n>1.
    - What if n=1?
      - minArray(a) = a[0]
    - What if n=0?
      - minArray is not defined

Intuition behind recursion

- minArray(a) = Not defined if n=0
- minArray(a) = a[0] if n=1
- minArray(a) = Integer.min(minArray(a[0,....,n-2]), a[n-1]) otherwise
- Now comes the easiest part of recursion, coding the algorithm:

```
int minArray(int[] a, int n){
        if(n==0) throw new RuntimeError(“Min of empty array is not defined.”);
        if(n==1) return a[0];
        return Integer.min(a[n-1], minArray(a, n-1));
}
```

Intuition behind recursion

- Given the minimum of the array a_left = 101, 53, 23, 65, is **x_left** and the minimum of the array a_right = 28, 97, 14, 83 is **x_right**; Can we find the minimum of a = 101, 53, 23, 65, 28, 97, 14, 83 ?
  - Yes. Integer.min(x_left, x_right)
  - => minArray(a) = min(minArray(a[0,....,n/2]), a[n/2+1,.....,n-1])
    - Is this correct?

Intuition behind recursion

- Given the minimum of the array a_left = 101, 53, 23, 65, is **x_left** and the minimum of the array a_right = 28, 97, 14 is **x_right**; Can we find the minimum of a = 101, 53, 23, 65, 28, 97, 14, 83 ?
  - Yes. Integer.min(x_left, x_right)
  - => minArray(a) = min(minArray(a[0,....,n/2]), a[n/2+1,.....,n-1])
    - Is this correct? No. **Base case!**
    - minArray(a) = a[0] if n=1 and Integer.min(a[0], a[1]) if n=2

Today

1. Stack
2. Homework2
3. Linked Lists
    a. Singly Linked Lists
    b. Doubly Linked Lists

Stacks

Stacks

- LIFO data structure

push(e):	Adds element e to the top of the stack.
pop( ):	Removes and returns the top element from the stack (or null if the stack is empty).
top( ):	Returns the top element of the stack, without removing it (or null if the stack is empty).
size( ):	Returns the number of elements in the stack.
isEmpty( ):	Returns a boolean indicating whether the stack is empty.

Source: Goodrich, Tamassia, and Goldwasser

Stacks

- Common implementations:
  - Array
  - Linked List

Source: Goodrich, Tamassia, and Goldwasser

Stacks: array-based implementation

Pros

- Simple
- Efficient (O(1) runtimes for top() and pop())

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
top	$O(1)$
push	$O(1)$
pop	$O(1)$

Cons

- Arrays have a fixed capacity

Source: Goodrich, Tamassia, and Goldwasser

Arrays: (singly) linked list-based implementation

size( ):	Returns the number of elements in the list.
isEmpty( ):	Returns **true** if the list is empty, and **false** otherwise.
first( ):	Returns (but does not remove) the first element in the list.
last( ):	Returns (but does not remove) the last element in the list.
addFirst(e):	Adds a new element to the front of the list.
addLast(e):	Adds a new element to the end of the list.
removeFirst( ):	Removes and returns the first element of the list.

Arrays: linked list-based implementation

Stack Method	*Singly Linked List Method*
size( )	list.size( )
isEmpty( )	list.isEmpty( )
push(e)	list.addFirst(e)
pop( )	list.removeFirst( )
top( )	list.first( )

Practice

Create a recursive and iterative method to remove all elements from the stack.

```
public void recursiveRemove()

public void iterativeRemove()
```

Source: Goodrich, Tamassia, and Goldwasser

```
  public void recursiveRemoveAll() {
  if (isEmpty()) {
    return;
  } else {
    System.out.println(pop());
    recursiveRemoveAll();
  }
}

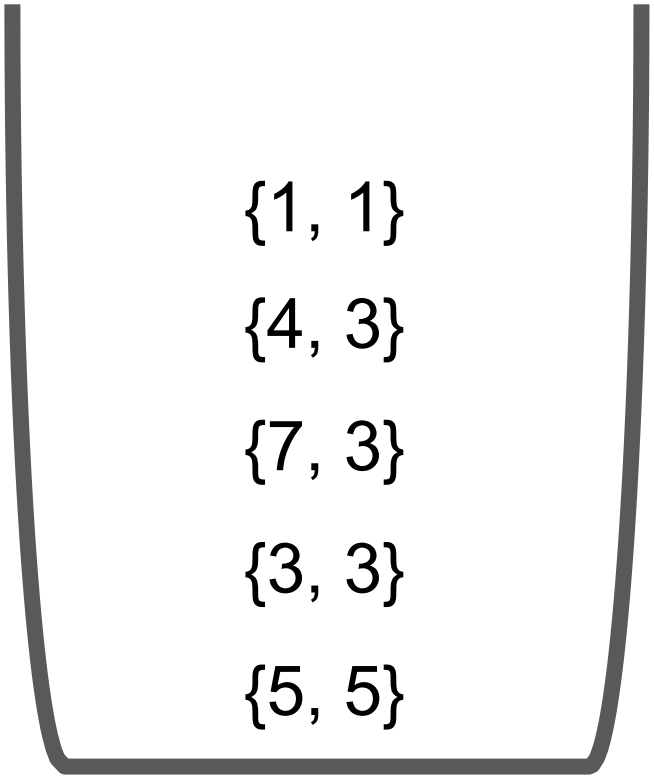
public void iterativeRemoveAll() {
  while (!isEmpty()) {
    System.out.println(pop());
  }
}
```

Practice

Design a stack that supports push, pop, top, and retrieving the minimum element in **constant time**.

- push(x) - Push element x onto stack.
- pop() - Removes the element on top of the stack.
- top() - Get the top element.
- getMin() - Retrieve the minimum element in the stack.

Source: Leetcode



{5, 5}	{3, 3}	{7, 3}	{4, 3}	{1, 1}

Homework

Homework 2: Postfix Calculator

Postfix notation is a **notation** for writing arithmetic expressions in which the operands appear before their operators.

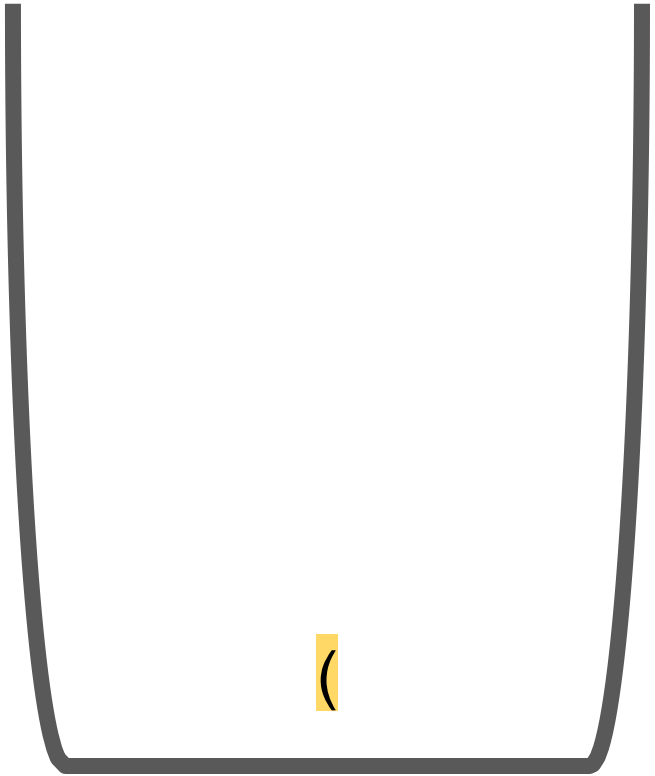
$$3 + 4 \longrightarrow 3\ 4\ +$$

Example

Infix expression: (4+8)*(6-5)/((3-2)*(2+2))

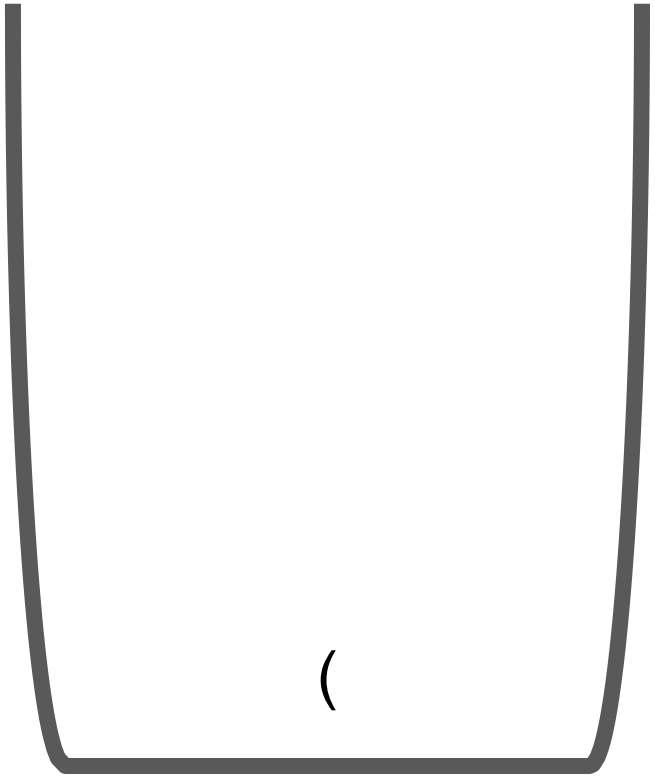
Postfix expression: 4 8 + 6 5 - * 3 2 - 2 2 + * /

(4+8)*(6-5)/((3-2)*(2+2))



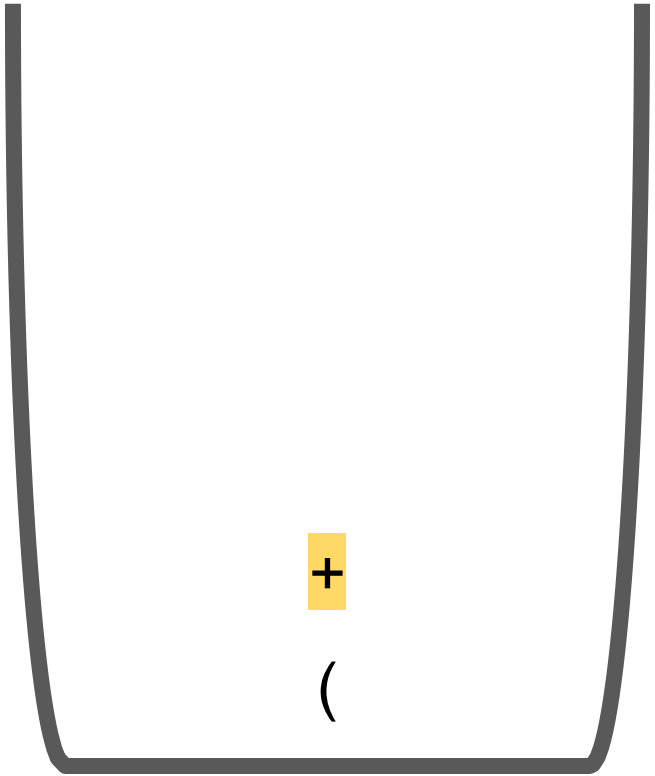
4+8)*(6-5)/((3-2)*(2+2))

4



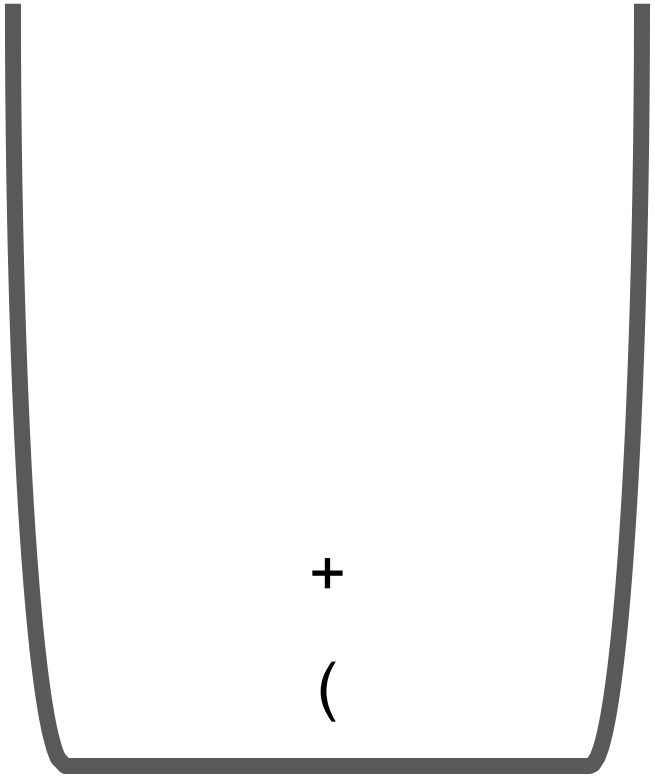
+8)*(6-5)/((3-2)*(2+2))

4



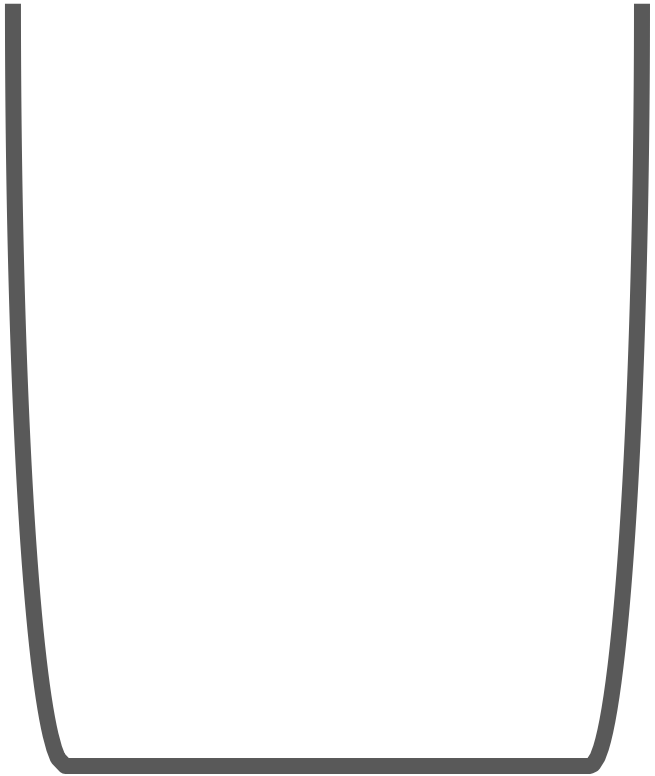
8)*(6-5)/((3-2)*(2+2))

4 8



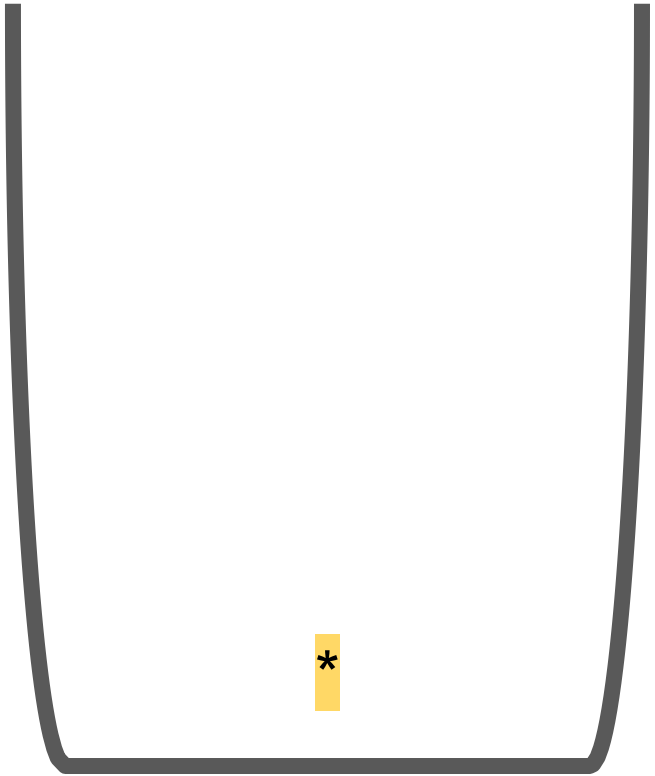
)*(6-5)/((3-2)*(2+2))

4 8 +



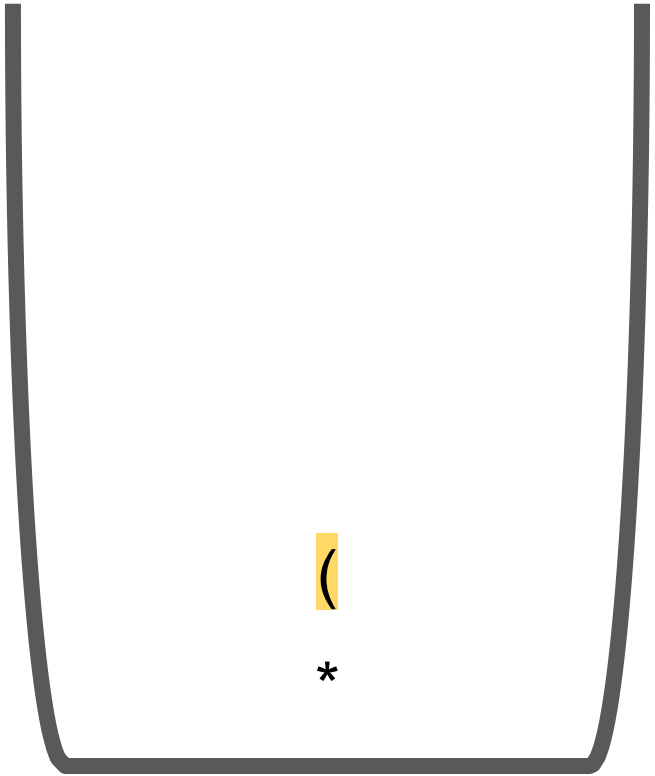
(6-5)/((3-2)(2+2))

4 8 +



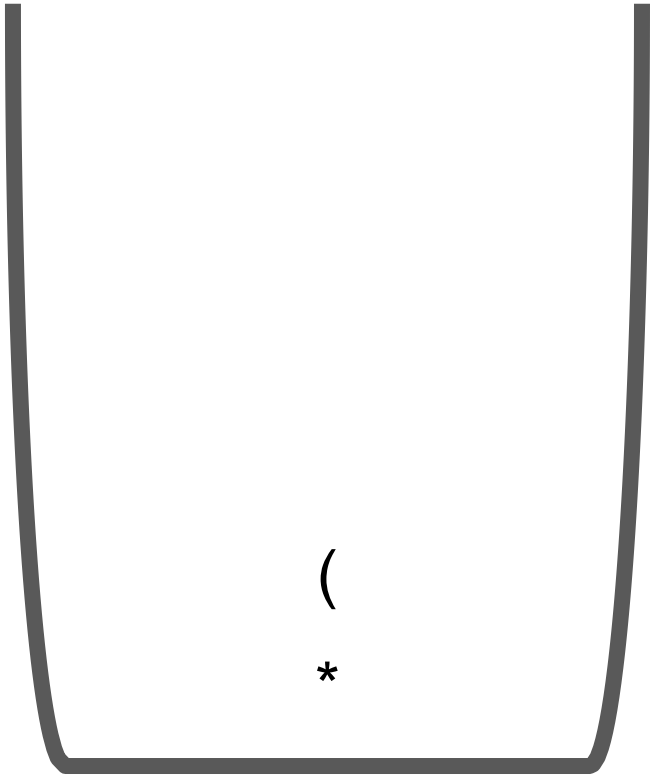
(6-5)/((3-2)*(2+2))

4 8 +



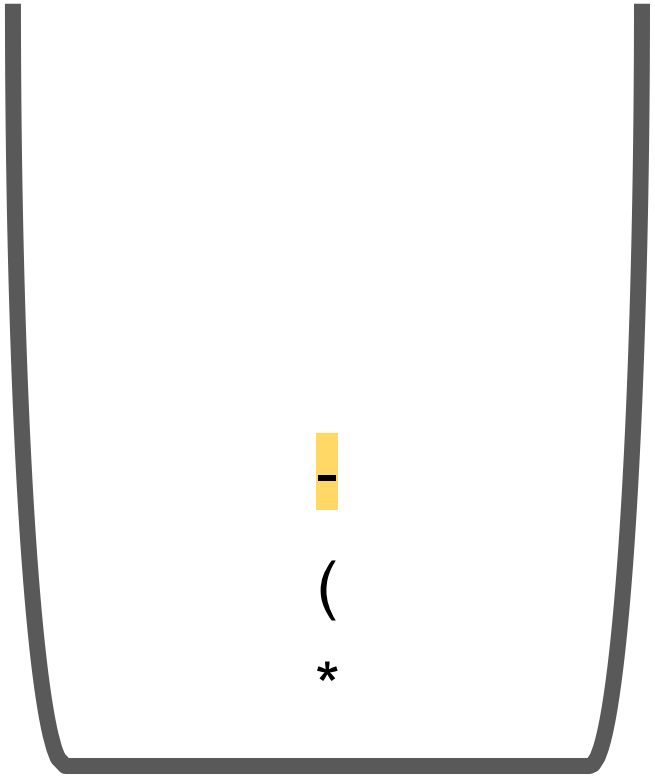
6-5)/((3-2)*(2+2))

4 8 + 6



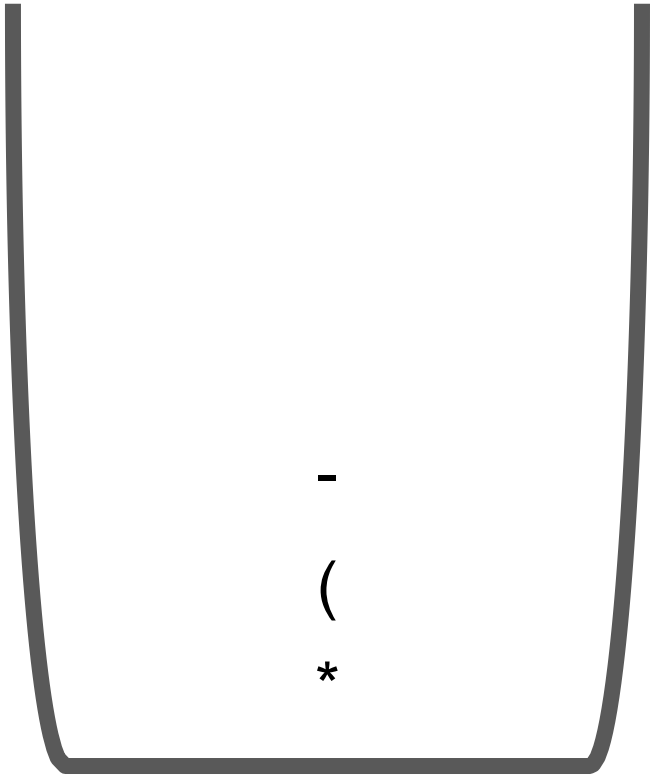
-5)/((3-2)*(2+2))

4 8 + 6



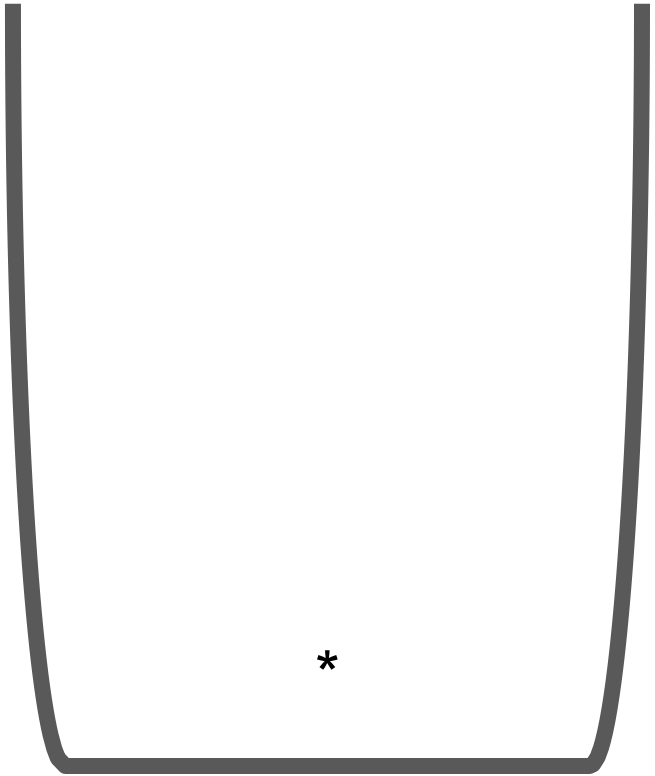
5)/((3-2)*(2+2))

4 8 + 6 5



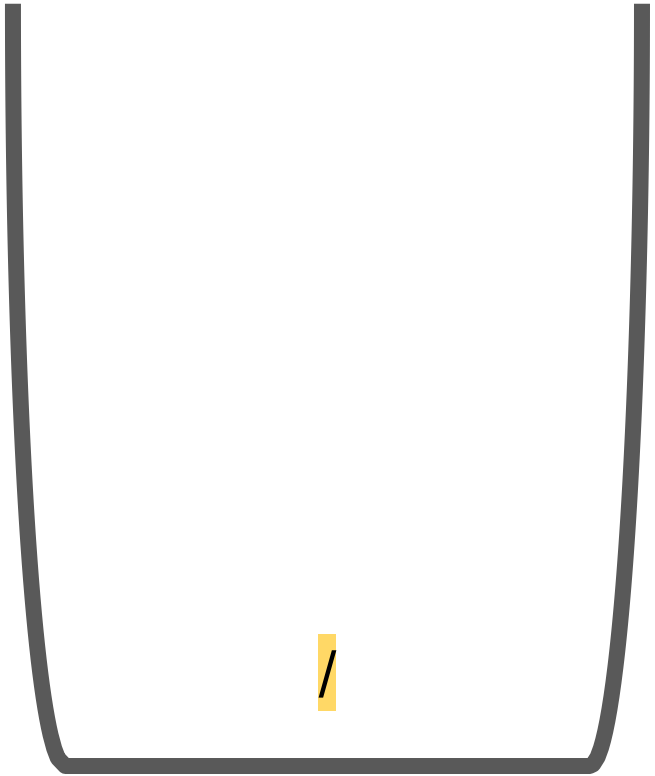
)/((3-2)*(2+2))

4 8 + 6 5 -



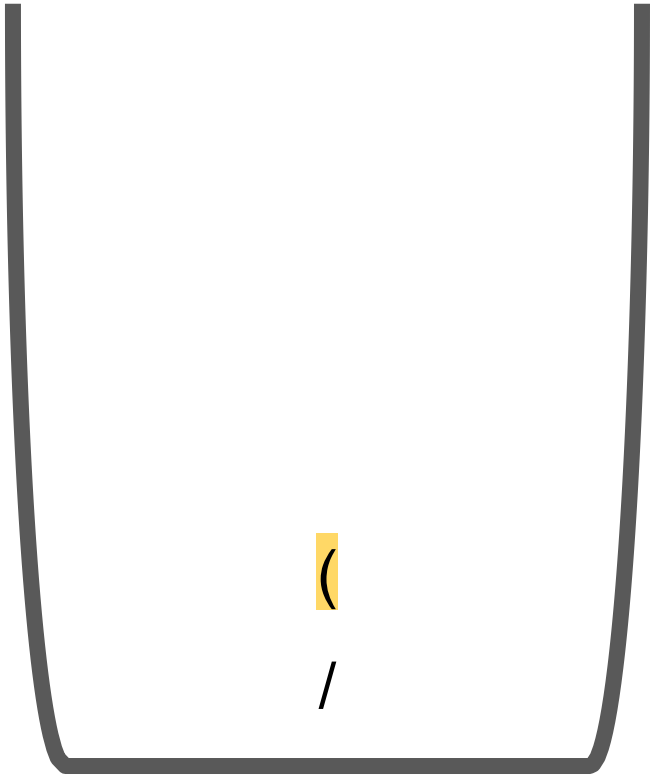
/((3-2)*(2+2))

4 8 + 6 5 - *



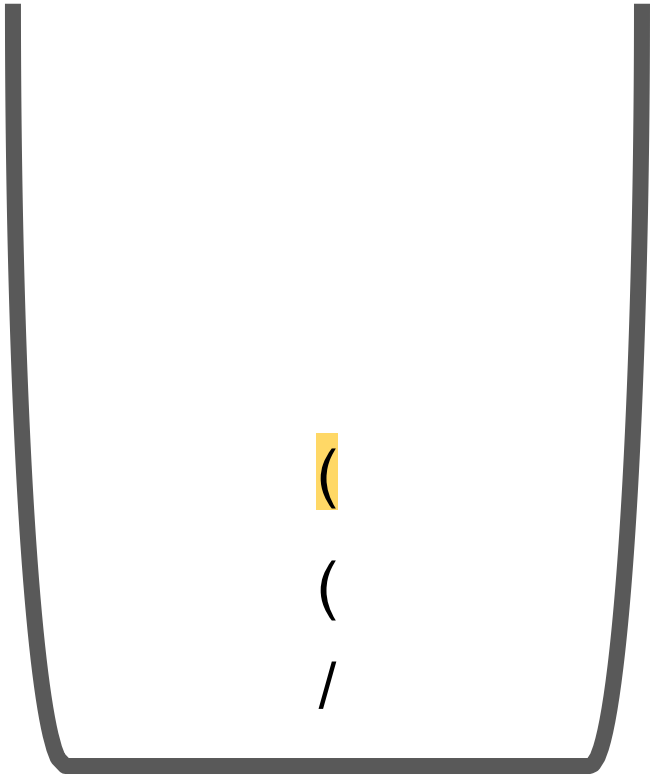
((3-2)*(2+2))

4 8 + 6 5 - *



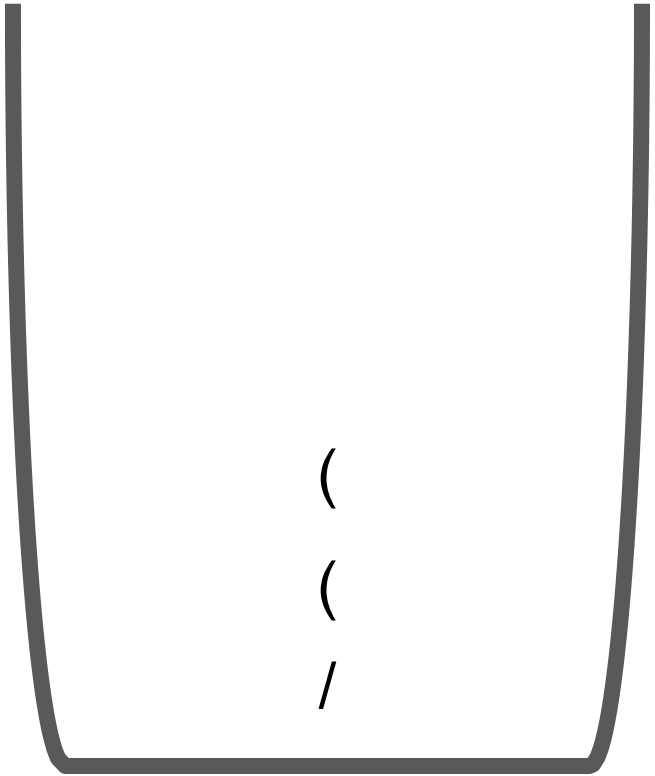
(3-2)*(2+2))

4 8 + 6 5 - *



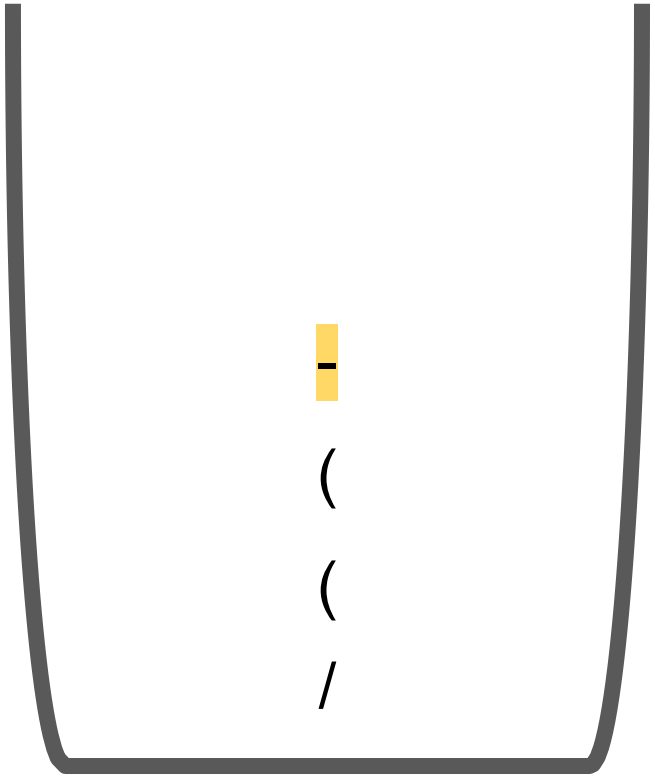
3-2)*(2+2))

4 8 + 6 5 - * 3



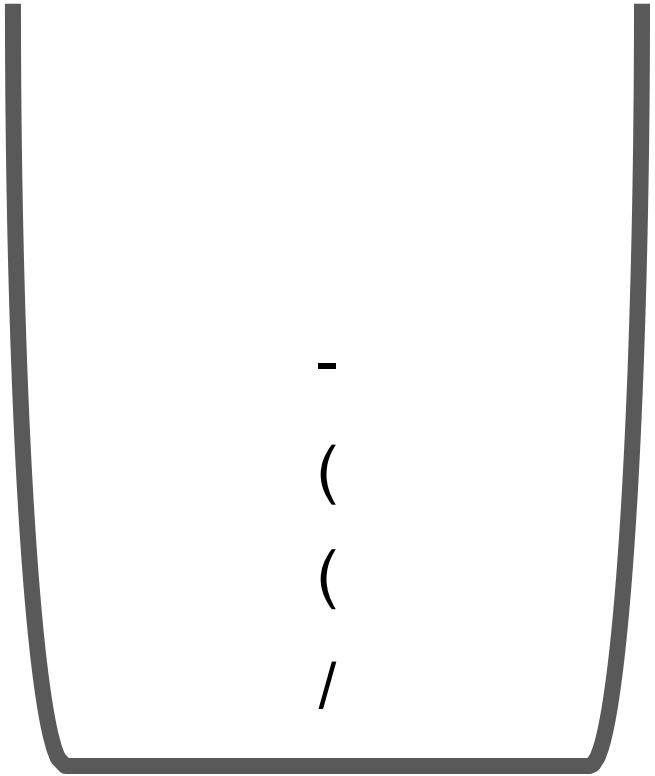
-2)*(2+2))

4 8 + 6 5 - * 3



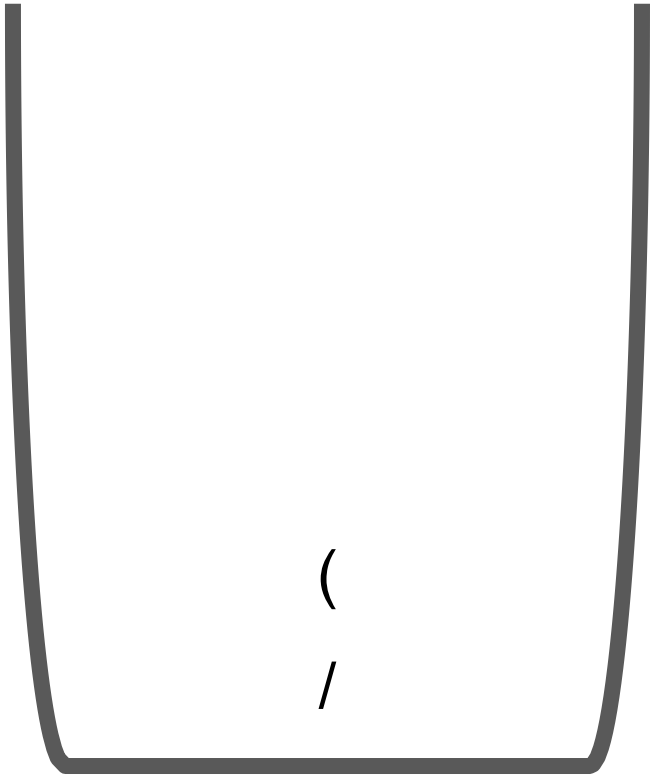
2)*(2+2))

4 8 + 6 5 - * 3 2



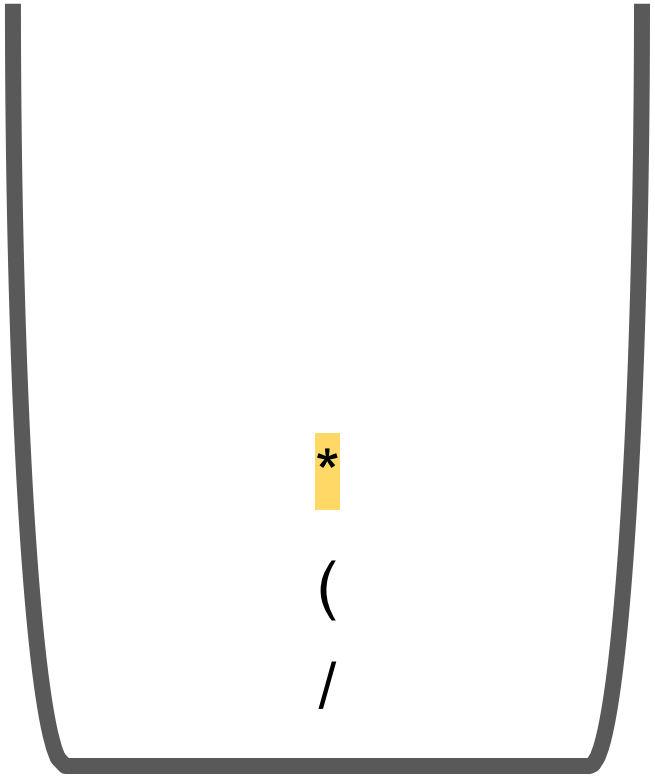
)*(2+2))

4 8 + 6 5 - * 3 2 -



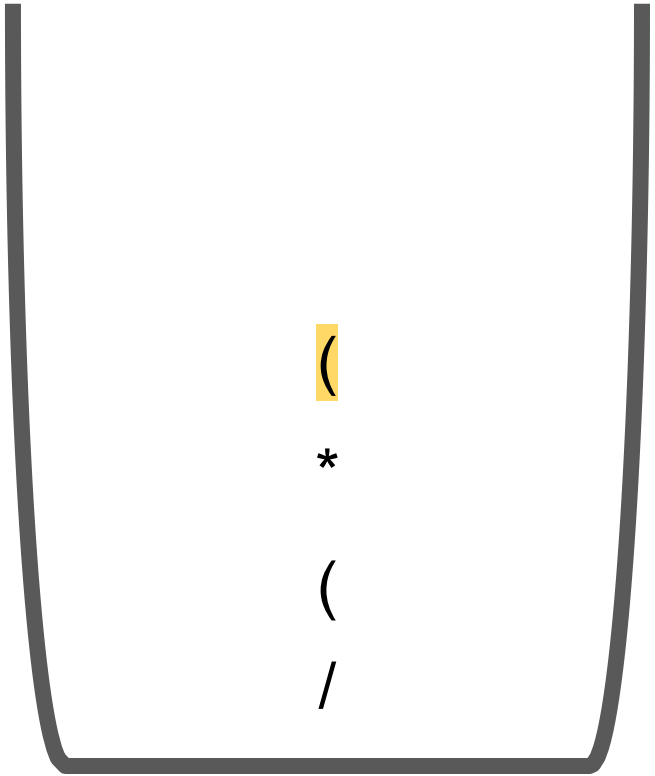
*(2+2))

4 8 + 6 5 - * 3 2 -



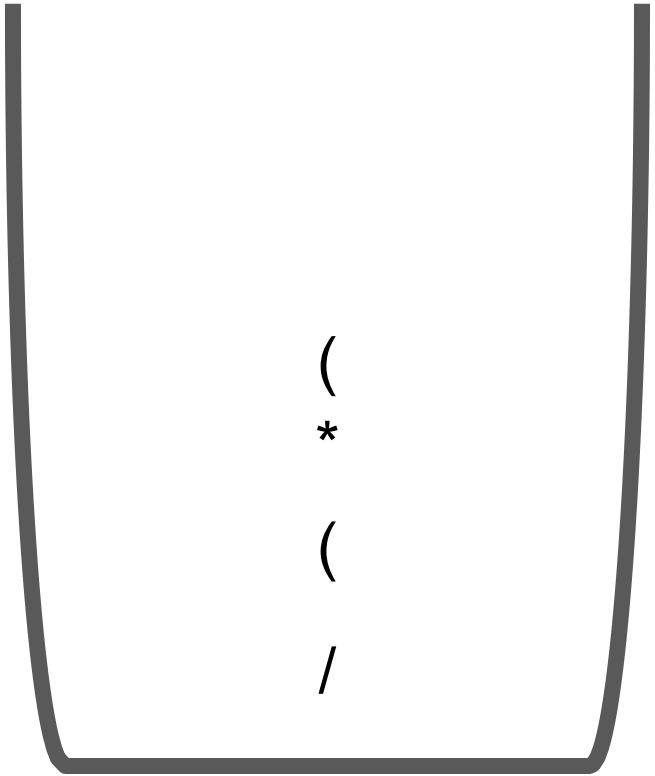
(2+2))

4 8 + 6 5 - * 3 2 -



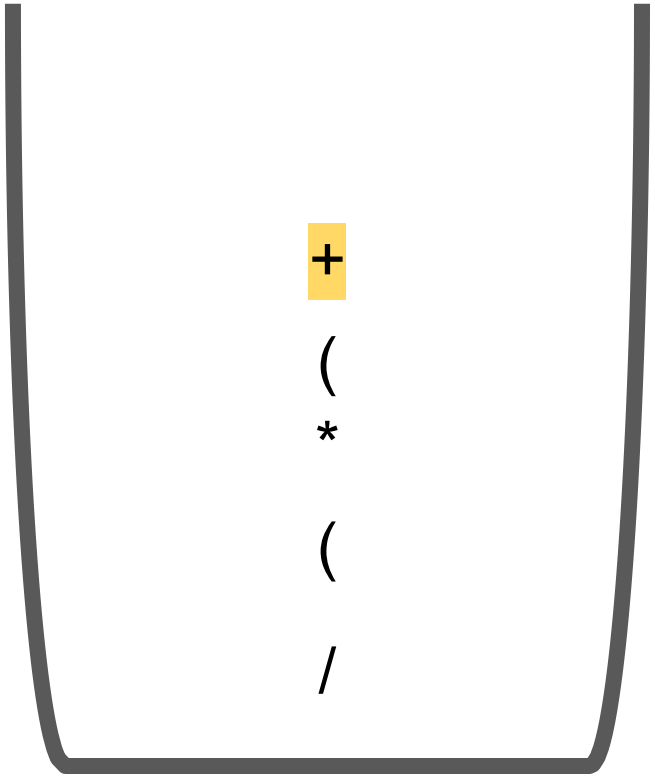
2+2))

4 8 + 6 5 - * 3 2 - 2



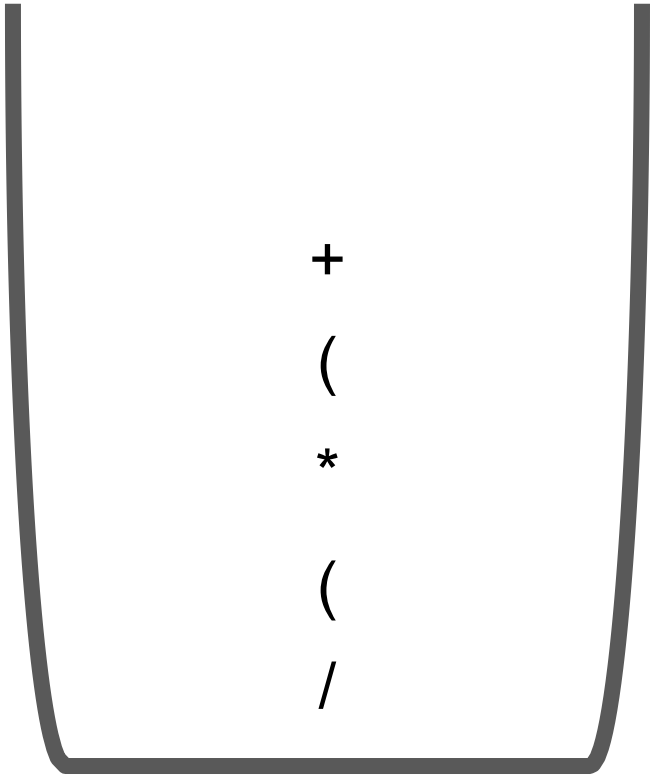
+2))

4 8 + 6 5 - * 3 2 - 2



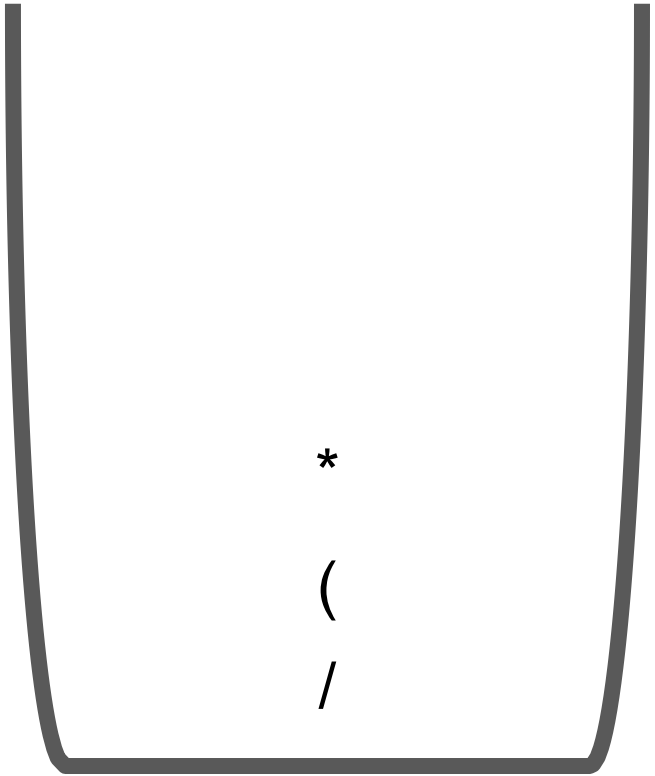
2))

4 8 + 6 5 - * 3 2 - 2 2



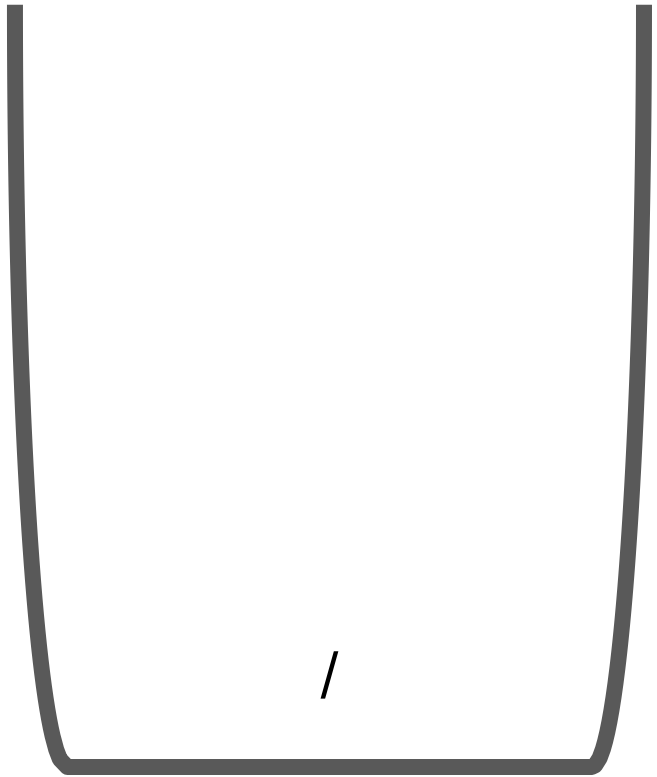
))

4 8 + 6 5 - * 3 2 - 2 2 +

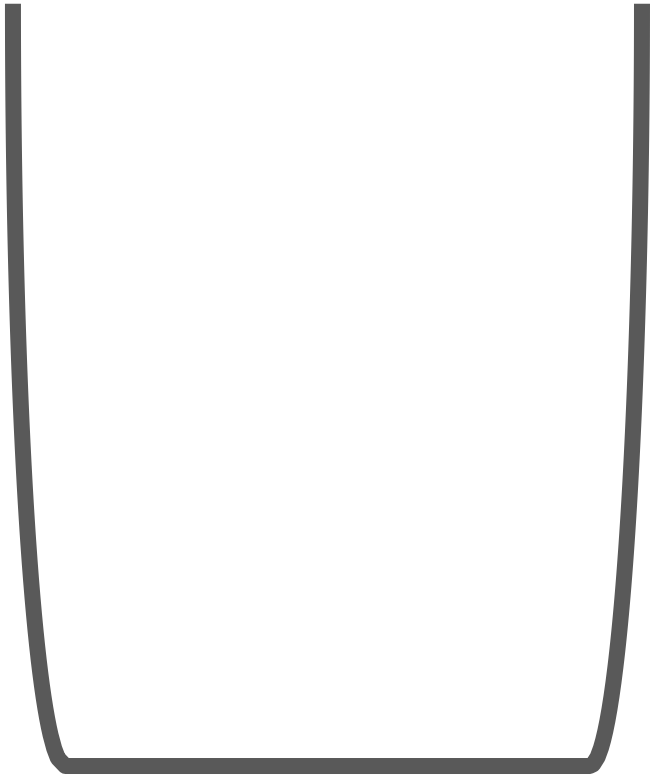


)

4 8 + 6 5 - * 3 2 - 2 2 + *



4 8 + 6 5 - * 3 2 - 2 2 + * /



Singly Linked Lists

SinglyLinked List ADT

size( ):	Returns the number of elements in the list.
isEmpty( ):	Returns **true** if the list is empty, and **false** otherwise.
first( ):	Returns (but does not remove) the first element in the list.
last( ):	Returns (but does not remove) the last element in the list.
addFirst(e):	Adds a new element to the front of the list.
addLast(e):	Adds a new element to the end of the list.
removeFirst( ):	Removes and returns the first element of the list.

Source: Goodrich, Tamassia, and Goldwasser

Practice: Singly Linked Lists

(C-3.25) Describe an algorithm for concatenating two singly linked lists *L* and *M*, into a single list *L′* that contains all the nodes of *L* followed by all the nodes of *M*.

Source: Goodrich, Tamassia, and Goldwasser

C-3.25 Solution

```
//clones list
public SinglyLinkedList<E> clone() {
	SinglyLinkedList<E> cpy = new SinglyLinkedList<>();
	Node<E> curr = this.head;
	while (curr!=null) {
		cpy.addLast(curr.getElement());
		curr = curr.getNext();
	}
	return cpy;
}

//concatenates passed in list to current list
public SinglyLinkedList<E> concatenate(SinglyLinkedList<E> m) {
	if (this.isEmpty()) {
		return m.clone();
	} else if (m.isEmpty()) {
		return this.clone();
	}
	else {
		SinglyLinkedList<E> lm = this.clone();
		SinglyLinkedList mCopy = m.clone();
		lm.tail.setNext(mCopy.head);
		return lm;
		}
	}
```

Doubly Linked Lists

Singly linked lists: limitations

- Limitations that stem from the asymmetry of a singly linked lists:
  - Unable to efficiently delete a node from the tail of the list
  - More generally, we are unable to efficiently delete arbitrary nodes from interior positions if only given a reference to that node

Source: Goodrich, Tamassia, and Goldwasser

Doubly Linked Lists

- Each node keeps an explicit reference to the node before it ***and*** a reference to the node after it
- Allows a greater variety of O(1) time update operations, including insertions and deletions at arbitrary positions within the list

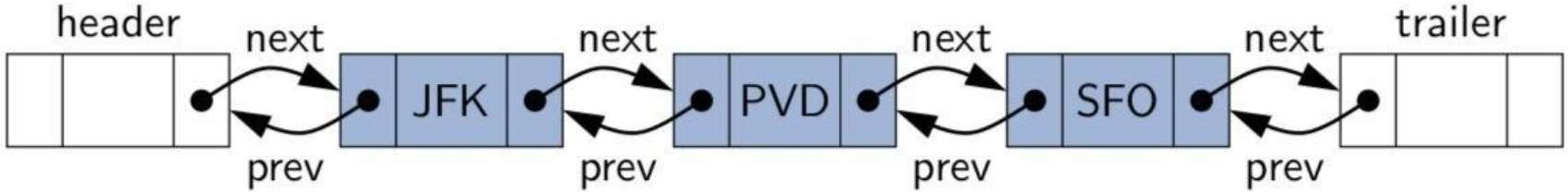
Source: Goodrich, Tamassia, and Goldwasser

Header and trailer sentinels

- “Dummy” nodes / sentinel nodes
- Do not store elements of the primary sequence
- Advantages
  - All insertions can be treated in a unified manner, because a new node will always be placed between a pair of existing nodes
  - Every element that is to be deleted is guaranteed to be stored in a node that has neighbors on each side

Source: Goodrich, Tamassia, and Goldwasser

Header and trailer sentinels



Source: Goodrich, Tamassia, and Goldwasser

Doubly Linked List

- We will provide a private method, `addBetween`, to handle the general case of an insertion, and then we will rely on that utility as a straightforward method to implement both `addFirst` and `addLast`.
- In similar fashion, we will define a private `remove` method that can be used to easily implement both `removeFirst` and `removeLast`.

Source: Goodrich, Tamassia, and Goldwasser

Doubly Linked List

```
private void addBetween(int data, Node predecessor, Node
successor)
```

```
private Integer remove(Node node)
```

Practice: Doubly Linked Lists

(R-3.8) Describe a method for finding the middle node of a doubly linked list with header and trailer sentinels by “link hopping,” and without relying on explicit knowledge of the size of the list. In the case of an even number of nodes, report the node slightly left of center as the “middle.” What is the running time of this method?

Source: Goodrich, Tamassia, and Goldwasser

R-3.8 Solution

```
private Integer middle () {
   if (isEmpty()) return null;

   Node middleNode = head.next;

   Node parterNode = tail.prev;

   while (middleNode != partnerNode && middleNode.next != partnerNode) {

      middleNode = middleNode.next;


      partnerNode = patnerNode.prev;

   }

   return middleNode.data;

}
```